

Proyecto POO — Tienda de productos

Explicación de los diagramas del sistema

Este documento explica los tres artefactos de diseño que precedieron a la implementación. Los tres describen el mismo sistema visto desde tres ángulos complementarios: **las clases** (visión POO), **la base de datos** (visión de datos) y **la arquitectura** (visión de capas). Cada sección adjunta el diagrama correspondiente junto con su explicación.

1. Diagrama de clases UML

Fuente: *diagrama generado con PlantUML (plantuml.com) — Diagrama de clases UML.png*

El diagrama muestra las cinco clases que forman el Modelo del sistema. Es la pieza central del diseño porque es la evidencia directa de los conceptos de POO pedidos en el enunciado. Cada clase se dibuja como una caja con tres zonas: nombre de la clase arriba, atributos en el centro (con signo - privado o + público) y métodos abajo.

Las cinco clases y su responsabilidad

- **Categoria:** atributos privados *id*, *nombre*, *descripcion*; métodos de acceso y *es_valida()*. Agrupa los productos del catálogo.
- **Producto:** *id*, *nombre*, *descripcion*, *precio_compra*, *precio_venta*, *stock* y referencia a su *categoria*. Métodos numéricos (*calcular_ganancia()*, *aplicar_descuento(porcentaje)*), lógicos (*hay_stock_disponible(cantidad)*) y mutador con efecto (*actualizar_stock(cantidad)*). Destructor *__del__()*.
- **Cliente:** *id*, *nombre*, *email*, *telefono*, *puntos*. *es_valido()*, *es_cliente_frecuente()* y *acumular_puntos(monto)*.
- **Pedido:** referencia a un objeto *Cliente* y lista de objetos *DetallePedido*. Métodos de composición *agregar_producto(producto, cantidad)* y *confirmar()*; *calcular_total()* aplica descuento a clientes frecuentes; *cambiar_estado(nuevo_estado)* valida transiciones.
- **DetallePedido:** referencia a un objeto *Producto* con *cantidad* y *precio_unitario*; *calcular_subtotal()*. Une Pedido con Producto.

Relaciones representadas

Relación	Significado
Categoria 1 — 0..* Producto	Una categoría agrupa de cero a muchos productos
Producto 1 — 0..* DetallePedido	Un producto figura en muchas líneas de pedido
Pedido 1 — 1..* DetallePedido	Un pedido contiene una o más líneas
Cliente 1 — 0..* Pedido	Un cliente realiza de cero a muchos pedidos



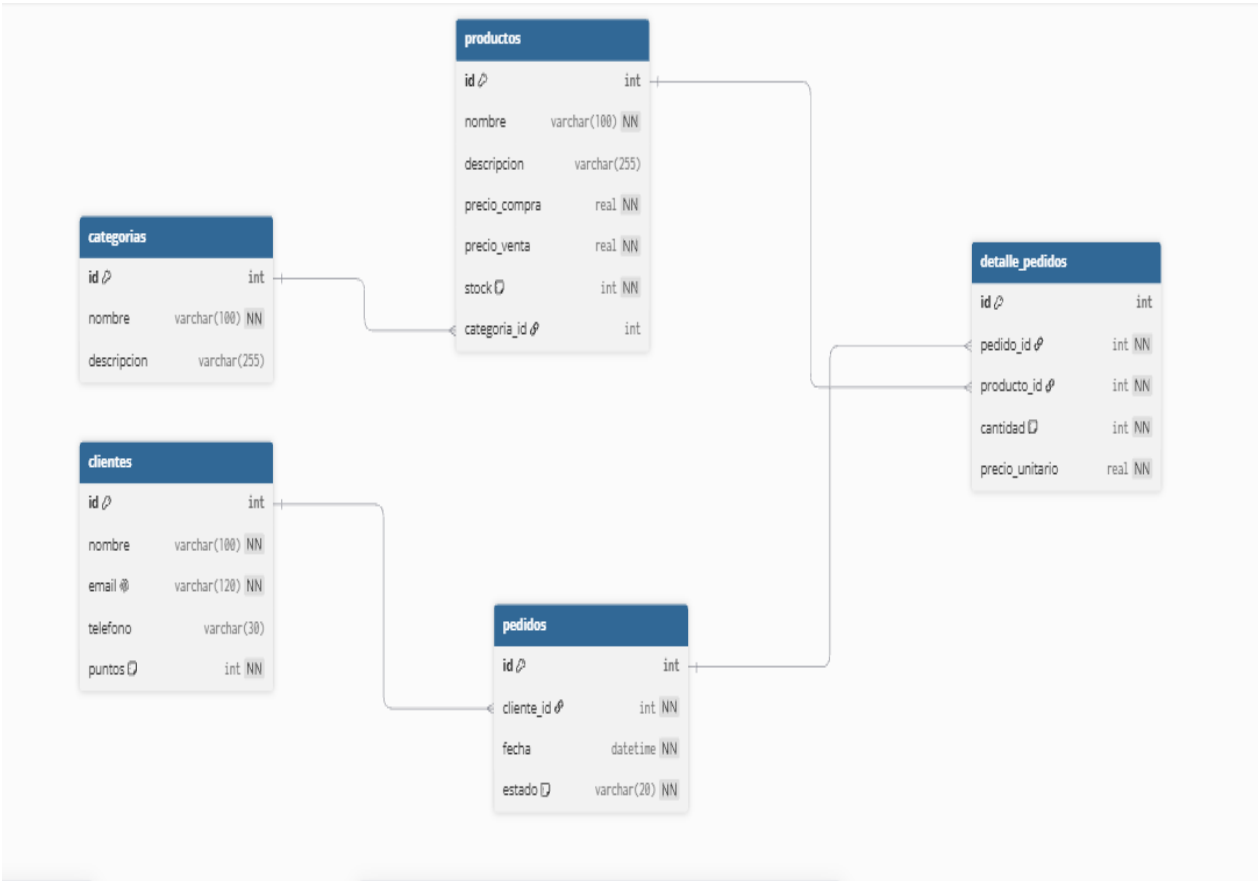
2. Diagrama de la base de datos

Fuente: *diagrama generado con dbdiagram.io (DBML) — Diagrama de la base de datos.png*

Representa la persistencia real del sistema en SQLite: cinco tablas que son la traducción en datos de las cinco clases del diagrama anterior. Las llaves primarias (PK) y foráneas (FK) muestran los enlaces entre tablas.

Tabla	Campos clave	Relación
categorias	id, nombre, descripcion	origen de la FK de productos
productos	id, nombre, descripcion, precio_compra, precio_venta, stock, categoria_id	categoria_id apunta a categorias.id
clientes	id, nombre, email (único), telefono, puntos	origen de la FK de pedidos
pedidos	id, cliente_id, fecha, estado	cliente_id apunta a clientes.id
detalle_pedidos	id, pedido_id, producto_id, cantidad, precio_unitario	FK a pedidos.id y productos.id

La columna *puntos* de *clientes* alimenta el método *es_cliente_frecuente()*, y la columna *estado* de *pedidos* se controla desde la clase *Pedido*.

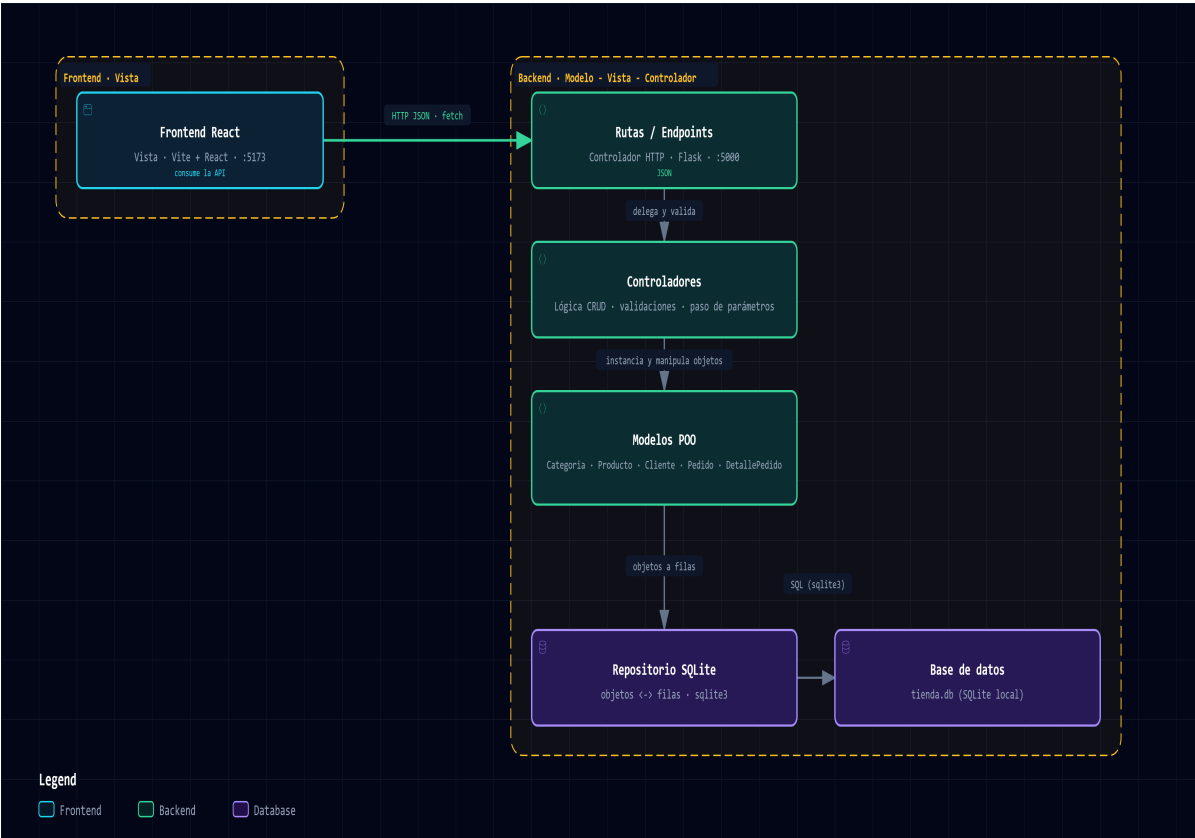


3. Diagrama de arquitectura (MVC)

Fuente: *diagrama generado con archify — Diagrama de arquitectura.png*

Representa el sistema completo en tres capas MVC: la Vista (Frontend React) consume la API; el Controlador (rutas Flask) recibe las peticiones y responde JSON; los Modelos POO resuelven la lógica de negocio; el Repositorio SQLite traduce objetos a filas y viceversa, y la Base de datos (*tienda.db*) persiste localmente.

Lectura: la petición viaja desde la Vista hacia el backend, atraviesa la capa de Controlador, se resuelve en los Modelos y termina en la base de datos. Este diagrama es la evidencia del requisito de arquitectura MVC y de ampliabilidad: cada capa se puede modificar o crecer sin tocar las demás.



4. Cómo se relacionan los tres diagramas

- **Diagrama de clases:** qué objetos existen y qué pueden hacer (visión de código).
- **Diagrama de base de datos:** dónde y cómo se guardan esos objetos (visión de datos).
- **Diagrama de arquitectura:** en qué capa vive cada pieza y cómo fluye una petición (visión de sistema).

La coherencia entre los tres es la que permite rastrear un cambio de diseño —por ejemplo, agregar una nueva categoría o un nuevo estado de pedido— desde la clase hasta la tabla y hasta la capa correspondiente sin romper el resto.